OPSD source audit against design v1.2 (2026-10-26)

What was read. The OPSD repository files `opsd_trainer.py`, `opsd_train.py`, `data_collator.py`, `scripts/run_opsd_1b.sh`, `scripts/run_opsd_4b.sh`, `scripts/run_opsd_8b.sh`, `scripts/run_opsd_8b_nonthink.sh` and the README, all at `github.com/siyan-zhao/OPSD` (branch `main`), read on 2026-10-26.

How it was read. The files were read through a fetch tool that returns exact short quotes plus summaries. Short exact quotes are given below in `code` style. Everything else is a paraphrase of the code.

What could not be read. OP-Print (arXiv 2610.31616) and the HuggingFace Qwen3 tokenizer config could not be re-read in this session: `arxiv.org`, `alphaxiv.org` and `huggingface.co` were unreachable. Statements about OP-Print's own setup are therefore still **[inference]** from the match with the official scripts (see v1.1 §v0 → v1, item 1).

Markers. As in the design: **[verified]** means read in the code here; **[inference]** and **[unverified]** as before.

1. Facts in v1.2 that the code confirms

v1.2 claim	Code	Status
β = 0 summand is p_k(log p_k − log q_k)	`F.kl_div(student_log_probs, teacher_log_probs, reduction="none", log_target=True)`	verified
Clip is element-wise, before the vocabulary sum	`jsd = jsd.clamp(max=token_clip)`, then `jsd = jsd[mask]`, then `jsd.sum() / mask.sum()`	verified
clip = 0 disables the clip	The trainer gates only on `if token_clip is not None:`. `opsd_train.py` passes `jsd_token_clip=script_args.jsd_token_clip if script_args.jsd_token_clip > 0 else None`.	verified, **through the launcher only** (§3.6)
No `.float()` before `log_softmax`	No `.float()`, `.to(torch.float32)` or autocast anywhere in `opsd_trainer.py`; `student_logits_for_loss = student_logits`	verified (logit dtype still depends on the `transformers` version)
Fixed teacher = adapter-disabled base under no-grad	`adapter_context = ...disable_adapter()`; `with torch.no_grad(), adapter_context:`	verified
merge → load_weights → unmerge at vLLM sync	`self.model.merge_adapter()`, `llm_model.load_weights(...)`, `self.model.unmerge_adapter()`; no fp32 copy of the base is kept	verified
Clip 0.05 (1.7B, 4B), 0.06 (8B); LoRA r = 64, α = 128, all 7 projections; lr 5e-6; max grad norm 0.1; T = 1.1, top-p 0.95, top-k 20; β = 0; 1024-token completions	`run_opsd_1b.sh`, `run_opsd_4b.sh`, `run_opsd_8b.sh`	verified
README: style tokens show "6–15×" higher KL; the clip prevents them from "dominating"; the loss can be negative	README	verified
Non-thinking result tables use clip 1e-6 (1.7B, 4B) and 1e-7 (8B)	README; `run_opsd_8b_nonthink.sh` has `--jsd_token_clip 1e-7`	verified
The tinker-style loss is unclipped	`loss = -(advantage * student_log_probs_sampled_masked).mean()` with `advantage = (teacher_log_probs_sampled - student_log_probs_sampled).detach()`; no clamp	**verified** (v1.2 had [inference])

v1.2 claim	Code	Status
`--top_k_loss` renormalises both distributions over the teacher's top-k	Docstring and code: top-k indices from teacher logits, `log_softmax` over the k gathered entries	verified
Dataset	`load_dataset("siyanzhao/Openthoughts_math_30k_opsd")`, fields `problem` and `solution`	verified (new detail)

The arithmetic in v1.2 was also recomputed and is correct:

- adapter size 69,730,304 (r = 64) and 8,716,288 (r = 8);
- KV cache 114,688 B/token in bf16, ≈ 1.88 GB for 8k tokens in fp32;
- one 64 × V fp32 chunk = 38.9 MB;
- Hanley–McNeil SE of AUC 0.59 at 500/500 = 0.0179;
- the bf16 rounding prior ½·Var = 3.26e-4;
- Spearman–Brown(0.3) = 0.4615.

2. New facts the design does not yet use

2.1 The loss is computed at the sampling temperature

- `student_logits = student_logits / temperature` and `teacher_logits = teacher_logits / temperature` inside `generalized_jsd_loss`.
- The call passes `temperature=self.temperature`, with `self.temperature = args.temperature`.
- The same value is passed to vLLM's `SamplingParams`. In all scripts it is **1.1**.

Consequence. Training minimises Σ_k min(f_k, clip) with p = softmax(z_T/1.1) and q = softmax(z_S/1.1). The clamped set U, P_U, m_U and the clipped θ0-teacher arm (H1′ arm 3) must therefore all be computed at T = 1.1. Otherwise they do not describe the trained objective.

Whether OP-Print's attack Δ is computed at T = 1 or T = 1.1 is **[unverified]**: the paper could not be re-read. Both come from the same logits at no extra cost, so we score both. The analysis in `docs/results/` (R3) shows that T = 1.1 shrinks the clamped set relative to T = 1, but leaves the qualitative picture unchanged.

2.2 The 1.7B/4B/8B scripts run a thinking-mode teacher against a non-thinking student

- `opsd_train.py`: `student_thinking` has default `False`, with the help text "matches the main OPSD setup: student rolls out without ".
- `teacher_thinking` has default `True`, with the help text "Set to False for the matched non-thinking ablation (both nonthink)".
- `run_opsd_1b.sh`, `run_opsd_4b.sh` and `run_opsd_8b.sh` set neither flag. Only the `*_nonthink.sh` scripts set both to False.
- The collator passes `enable_thinking=self.student_thinking` to the student's `apply_chat_template` and `enable_thinking=self.teacher_thinking` to the teacher's. It appends or strips nothing afterwards.

[unverified here; from the Qwen3 chat template as we recall it] With `enable_thinking=False` the template ends the generation prompt with `<|im_start|>assistant\n<think>\n\n</think>\n\n`. With thinking enabled it ends at `<|im_start|>assistant\n`.

Consequences [inference]:

1. **Position t = 1.** The student generates after an empty think block. The teacher sees the same tokens appended directly after `assistant\n`, with no `<think>`. At t = 1 the teacher very likely puts most of its mass on `<think>`, which the student context makes near-impossible. Δ_1 can therefore be several nats in every sample, members and non-members alike.
    - The summand on `<think>` is far above any clip, so it is clamped. That position is essentially untrained under clip 0.05.
    - A large, roughly membership-independent spike in chunk 1 can make G-Max select chunk 1 in most samples. G-Max would then behave much like the "first 64 tokens" attack that v1.2 treats only as an ablation.
2. **Thinking-mode style.** For t > 1 the teacher is a thinking-mode model, and its user message also contains the transition prompt (§2.3). Both raise the teacher's probability on reflective "style" tokens. This is a candidate cause of the README's "6–15×" style-token KL that is unrelated to the content of r.
3. **Paper setup.** If OP-Print used these scripts unchanged (the [inference] in v1.1), its Δ contains both components. The matched ablation is one flag: `--teacher_thinking False`.

2.3 The teacher's user message adds instructions as well as r

With `reason_first=False` (the default, and not set by any script), the teacher user message is:

- `Problem: {problem}`;
- "Here is a reference solution to this problem:" with the solution between `=== Reference Solution Begin ===` and `=== Reference Solution End ===` markers;
- a transition prompt beginning "After reading the reference solution above, make sure you truly understand". It tells the model not to copy the reference but to re-derive the answer step by step, "exploring approaches and backtracking when needed";
- the same `\boxed{}` instruction as the student.

The student message is the problem plus the `\boxed{}` instruction.

Consequence. p_t and p_s differ in three ways: (a) r itself; (b) instruction text that explicitly asks for exploratory, backtracking style; (c) the thinking-mode template (§2.2).

Only (a) is sample-specific. (b) and (c) are shared by all samples, so a LoRA adapter can learn them *generically*. That reduces Δ for members and non-members alike, which adds Δ variance that carries no membership information.

Design implication (new, cheap). Add a **no-r teacher context** p_t\r: the same template, thinking flag and instructions, with the solution block removed. Then decompose:

- Δ^instr_θ0(x) = KL(p_θ0(·|p_t\r) ‖ p_θ0(·|p_s)) is the template-and-instruction part;
- Δ^r = Δ − Δ^instr is a heuristic r-specific part. KL is not additive, so the per-position KL(p_θ0(·|p_t) ‖ p_θ0(·|p_t\r)) is also logged.

This costs one extra θ0 prefill per sample. It also makes the §1.0 style/math diagnostic interpretable: high style-token Δ that persists without r is not a reference-correction signal.

2.4 Mid-sequence padding at t = 1 in training

- The collator forces `self.tokenizer.padding_side = "right"` and pads teacher prompts to the micro-batch maximum.
- The trainer builds `teacher_full_ids = torch.cat([teacher_prompts, generation_ids], dim=1)`, masks pad tokens (`teacher_attention_mask[teacher_full_ids == ...pad_token_id] = 0`), passes no

`position_ids`, and slices `outputs_teacher.logits[:, teacher_prompt_len - 1 : -1, :]` with one batch-level `teacher_prompt_len`.
- The student side is built the same way.

[inference] For every prompt shorter than the longest in its micro-batch (per-device batch 4 for 1.7B), the logit used for the **first generated token** is read at a pad position, not at the last prompt token. What that logit contains depends on the attention backend. With FlashAttention-2 unpadding, a masked position receives no attention output, so it is close to a context-free prediction.

Positions t ≥ 2 are read at real tokens, but with a RoPE gap equal to the number of pads.

Consequences.

- Training at t = 1 is partly or wholly ill-posed. This compounds §2.2.
- Our own scoring code must build every sequence without mid-sequence pads: per-sample prompts, or left padding with explicit `position_ids`.
- Δ_1 is reported separately, and G/G-Max are reported with and without t = 1.
- This is a property of the code we replicate, not something to fix in H0.

2.5 Optimisation budget: about 48 steps, 3 rollouts per member

- One completion per prompt per step: `n=1` ("In GKD, we generate 1 completion per prompt from student").
- `lmbda` is stored but never used. Generation is always on-policy.
- Effective batch is 32 prompts per optimiser step in every script:
  - 1.7B: 4 processes × 4 × gradient accumulation 2;
  - 4B: 8 × 4 × 1;
  - 8B: 8 × 2 × 2.
- At N = 500 and 3 epochs this gives **48 optimiser steps**, and each member contributes exactly **3 on-policy rollouts** to training.

Consequences.

- §1.1 must pin the **effective batch (32 prompts, 1 rollout each)**, not only the lr. On one 16 GB GPU this means gradient accumulation to 32. A smaller effective batch at the same lr and epochs gives more Adam steps and a larger adapter displacement, so it is a different operating point.
- The LT-IQR "3 points per sample" in v1.2 is exact.
- vLLM sync happens every `vllm_sync_frequency` steps (`state.global_step % self.trainer.vllm_sync_frequency == 0`). The default comes from TRL's `GOLDConfig` and was not read **[unverified]**. Pin it and log it. If it is > 1, rollouts come from a stale adapter.

2.6 Clip semantics depend on the entry point

- `OPSDTrainer(jsd_token_clip=0.0)` called directly (not via `opsd_train.py`) would clamp every positive summand at 0. That is the strongest possible clip, not "off".
- Our H3 harness must pass `None` for the clip = 0 arm, or go through `opsd_train.py`. An assertion is cheap: at clip = 0, the logged loss must equal the unclipped KL.

2.7 Other details

- Teacher prompts go through `truncation=True, max_length=self.max_length`. The scripts set `--max_length 20000`, so r is not truncated at the collator default of 2048.

- `reason_first` defaults to False in `opsd_train.py`; the collator's own default is True. No script sets it. EMA teacher is off (`use_ema_teacher: bool = False`).
- The README calls the flag `--use_tinker_loss`; the trainer's internal name is `use_thinking_machines_loss`. The objective is a sampled-token reverse-KL policy gradient. Temperature is applied there too.

3. Changes this audit implies for the design (carried into v1.3)

1. **§0 facts.** Add the loss temperature (T = 1.1), the thinking-mode asymmetry, the instruction asymmetry, the t = 1 padding issue, and the 48-step / 3-rollout budget.
2. **§1.0 diagnostic.** In addition:
   - log the chunk index of the argmax chunk and Δ_1;
   - report G/G-Max with t = 1 (and with chunk 1) excluded;
   - add the no-r teacher context p_t\r (§2.3).
3. **§1.1.** Pin effective batch 32, `vllm_sync_frequency` and `teacher_thinking`. The replication keeps the script defaults (student non-thinking, teacher thinking). `--teacher_thinking False` becomes a pre-registered secondary arm at clip 0.05 (2 models).
4. **H1/H1′.** Compute all clip-related quantities (U, P_U, m_U or its replacement, clipped D_θ) at T = 1.1.
5. **H3 harness.** Clip = 0 must be passed as `None`; add the loss-equality assertion.
6. **Scoring pipeline.** No mid-sequence pads (§2.4). Prefer per-sample sequences or left padding with explicit `position_ids`.